

SILLON — Guide d'installation administrateur

Ce guide décrit l'installation de SILLON sur un serveur cible, à destination des administrateurs de la direction. Il complète le [SILLON cahier des charges.md](#) (choix fonctionnels et techniques) et le [README.md](#) (vue d'ensemble, état du projet).

1. Prérequis

1.1 Système cible

- **Debian 13 (Trixie)**, architecture amd64.
- Accès réseau **limité aux dépôts Debian via le proxy d'entreprise** : aucun accès à internet en général, ni à un registre de conteneurs (Docker Hub ou équivalent). Cette contrainte est structurante — voir §4.
- Compte d'installation avec droits `root` (ou `sudo`).

1.2 Réseau

- Le serveur doit être joignable en HTTPS (443) depuis les postes des agents. Le port 80 est utilisé uniquement pour la redirection vers 443.
- Aucun port applicatif (API PostgREST sur 3000, orchestrateur sur 5000, PostgreSQL sur 5432) n'est exposé au-delà de `localhost` : tout le trafic externe transite par Nginx.

1.3 Paquets Debian requis

Les dépendances sont déclarées dans chaque paquet SILLON et installées automatiquement par `apt / dpkg` si le dépôt Debian est accessible : `postgresql-17`, `nginx`, `openssl`, `sudo`, `fail2ban`, `python3-flask`, `python3-psycopg2`, `python3-jwt`, `unicorn`, `podman`. Aucune action manuelle requise en amont.

2. Vue d'ensemble des paquets

Paquet	Rôle	Dépend de
<code>sillon-server</code>	Socle : PostgreSQL, Nginx, API de requêtage (PostgREST), catalogue applicatif, anti-bruteforce	—
<code>sillon-orchestrateur</code>	Création/suppression des bases, SQL libre, import CSV, partage, file d'attente	<code>sillon-server</code>
<code>sillon-worker</code>	Consommation de la file, lancement des conteneurs d'exécution de scripts	<code>sillon-orchestrateur</code>

Paquet	Rôle	Dépend de
<code>sillon-image-execution</code>	Image figée (Python/R) utilisée pour l'exécution des scripts	<code>sillon-worker</code>
<code>sillon-tutoriel</code> (<i>optionnel</i>)	Compte de démonstration, jeu de données réel et tutoriel PDF — VM de démo/formation uniquement, voir §7.2	<code>sillon-image-execution</code>
<code>sillon-demo-sirene</code> (<i>optionnel</i>)	Jeu de données Sirene massif (~43,9 millions de lignes) et scripts de démonstration à l'échelle — VM de démo/formation uniquement, voir §7.2	<code>sillon-tutoriel</code>

L'ordre d'installation ci-dessus est obligatoire : chaque paquet vérifie au `postinst` que le précédent est déjà configuré (présence de `/etc/sillon/secrets.env`, service actif) et refuse de s'installer sinon.

3. Obtention des paquets

Deux cas de figure :

- **Paquets déjà construits** : le dossier `build/` du dépôt contient les `.deb` prêts à l'emploi (`sillon-server_0.1.0_amd64.deb`, `sillon-orchestrateur_0.1.0_all.deb`, `sillon-worker_0.1.0_all.deb`, `sillon-image-execution_0.1.0_amd64.deb`, et les deux paquets optionnels `sillon-tutoriel_0.1.0_all.deb` et `sillon-demo-sirene_0.1.0_all.deb` — §7.2). C'est le cas le plus courant : passer directement à l'étape 4.
- **Reconstruction nécessaire** (nouvelle version, modification du code applicatif) : depuis une machine **ayant accès à internet** (le binaire PostgREST et l'image d'exécution sont téléchargés/construits à cette étape, pas sur la cible) :

```
bash cd build/ ./build.sh
```

Le script vendorise le binaire PostgREST officiel (vérifié par somme SHA256) et construit l'image Podman d'exécution, avant de produire les quatre `.deb`. Voir cahier des charges §12.2 à §12.5 pour le détail.

Transférer ensuite les quatre `.deb` vers le serveur cible (SCP, dépôt APT interne, ou tout autre moyen).

4. Pourquoi cette contrainte de vendoring ?

Point important à comprendre avant d'installer : le serveur cible **n'a jamais accès à un registre de conteneurs ni à internet**, seulement aux dépôts Debian via le proxy d'entreprise. Un `podman build` ou un téléchargement direct lancés au moment de l'installation échoueraient systématiquement (constaté lors des premiers tests de déploiement réels). C'est pourquoi :

- le binaire PostgREST est embarqué tel quel dans `sillon-server` (`/usr/lib/sillon/postgrest`) ;
- l'image d'exécution est construite en amont (hors cible) et embarquée en archive dans `sillon-image-execution` (`/usr/lib/sillon/image-execution.tar`), chargée localement via `podman load` au `postinst` — jamais reconstruite sur place.

Aucune action requise de l'administrateur sur ce point : c'est purement informatif, pour comprendre pourquoi une tentative de reconstruction sur la cible échouerait.

5. Installation

Sur le serveur cible, dans l'ordre :

```
sudo dpkg -i sillon-server_0.1.0_amd64.deb
sudo apt-get install -f      # résout les dépendances manquantes si besoin

sudo dpkg -i sillon-orchestrateur_0.1.0_all.deb
sudo dpkg -i sillon-worker_0.1.0_all.deb
sudo dpkg -i sillon-image-execution_0.1.0_amd64.deb
```

Sur une VM de démonstration ou de formation uniquement, deux paquets optionnels peuvent ensuite être installés (§7.2) :

```
sudo dpkg -i sillon-tutoriel_0.1.0_all.deb      # jamais sur un déploiement de
production
sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb   # optionnel, nécessite un accès
Internet - voir §7.2
```

Ce que fait chaque `postinst`, en résumé

- **sillon-server** : génère les secrets applicatifs (`/etc/sillon/secrets.env`, `chmod 600 root:root`), crée la base `sillon_catalog` et y déploie le catalogue (`schema.sql`), **dimensionne le moteur PostgreSQL selon la RAM et le nombre de cœurs détectés** (`shared_buffers`, `effective_cache_size`, `maintenance_work_mem`, parallélisation, points de contrôle, `pg_stat_statements` — §7.1, redéetecté et réappliqué sans effet si déjà correct à chaque réinstallation), configure l'API PostgREST (`/etc/sillon-api.conf`), génère un certificat TLS auto-signé si aucun n'est présent (`/etc/ssl/sillon/`), active le site Nginx, configure la limitation de débit et Fail2Ban sur `/api/rpc/login`, met en place la purge automatique du journal d'audit, et démarre les services `sillon-api` et `nginx`.

À la première installation, l'identifiant et le mot de passe administrateur initiaux s'affichent dans la sortie du `postinst` : `=== Identifiant administrateur initial : admin@sillon.local ===`
`Mot de passe administrateur initial : <généré aléatoirement> ===` **Noter ce mot de passe immédiatement** : il n'est affiché qu'une seule fois et n'est stocké nulle part en clair.

- **sillon-orchestrateur** : applique les droits sur `/opt/sillon-orchestrateur` , active et démarre le service `sillon-orchestrateur` (Gunicorn, port 5000 en local).
- **sillon-worker** : configure Podman en mode rootless pour l'utilisateur système `www-data` (répertoire `$HOME` dédié, gestionnaire de cgroups `cgroupfs` , plage subuid/subgid, lingering systemd), autorise la connexion PostgreSQL depuis les conteneurs de script (`pg_hba.conf`), puis active et démarre le service `sillon-worker` .
- **sillon-image-execution** : charge l'image d'exécution vendorisée dans le stockage Podman rootless de `www-data` .

Une mise à jour ultérieure (paquets déjà installés)

Les `postinst` détectent une installation existante (présence de la base `sillon_catalog`) et n'effectuent alors que des migrations incrémentales — **aucune recréation destructive**, secrets et certificat TLS institutionnel conservés tels quels.

6. Vérifications post-installation

```
# Services actifs
systemctl status sillon-api sillon-orchestrateur sillon-worker nginx postgresql
fail2ban

# Journaux en cas de problème
journalctl -u sillon-api -n 50
journalctl -u sillon-orchestrateur -n 50
journalctl -u sillon-worker -n 50
```

Puis, depuis un navigateur : se connecter à `https://<adresse-du-serveur>/` avec l'identifiant administrateur initial noté à l'étape 5, et changer immédiatement ce mot de passe depuis le panneau d'administration.

Le certificat TLS généré par défaut est **auto-signé** (donc affiché comme non fiable par le navigateur). Le remplacer par un certificat institutionnel avant toute mise en production réelle :

```
sudo cp mon-certificat.crt /etc/ssl/sillon/sillon.crt
sudo cp ma-cle.key /etc/ssl/sillon/sillon.key
sudo chown root:root /etc/ssl/sillon/sillon.crt
sudo chown root:www-data /etc/ssl/sillon/sillon.key
sudo chmod 644 /etc/ssl/sillon/sillon.crt
sudo chmod 640 /etc/ssl/sillon/sillon.key
sudo systemctl reload nginx
```

(Un `postinst` ultérieur ne régénère jamais ce certificat s'il est déjà présent — le remplacement ci-dessus est donc définitif jusqu'à intervention manuelle.)

7. Configuration optionnelle

7.1 Notifications par mail

Par défaut, l'orchestrateur tente d'envoyer les notifications de fin de traitement via un relais SMTP local (`localhost:25`), sans authentification. Pour utiliser un relais SMTP de la direction, ajouter les variables suivantes au service (`systemctl edit sillon-orchestrateur` , section `[Service]`) :

```
Environment=SILLON_SMTP_HOST=smtp.exemple-direction.fr
Environment=SILLON_SMTP_PORT=25
Environment=SILLON_SMTP_FROM=sillon@exemple-direction.fr
Environment=SILLON_URL=https://sillon.exemple-direction.fr
```

Puis `systemctl restart sillon-orchestrateur` . Un échec d'envoi de notification est journalisé (`journalctl -u sillon-orchestrateur`) mais ne bloque jamais le traitement lui-même.

7.2 Compte de démonstration et tutoriel

Deux façons de peupler un compte de démonstration, **jamais installées automatiquement avec le socle applicatif** — jamais sur un serveur de production :

- **sillon-tutoriel** (recommandé pour une VM de démonstration/formation) : cinquième paquet `.deb` , optionnel, à installer après les quatre autres (`sudo dpkg -i sillon-tutoriel_0.1.0_all.deb`). Son `postinst` crée (ou réutilise) le compte `demo@sillon.local` / `demo` (profil agent) et importe un **jeu de données réel** : les 34 868 communes de France et leurs 18 régions (source data.gouv.fr, INSEE/IGN, Licence Ouverte 2.0), avec des requêtes d'exemple déjà dans l'historique et deux scripts Python/R déjà exécutés. Fournit en plus un **tutoriel PDF complet** (exercices SQL en difficulté croissante, puis formation avancée Python et R), installé dans `/usr/share/doc/sillon-tutoriel/` et publié aux côtés de la documentation existante sur `https://<serveur>/Documentation/` . `sudo apt-get purge sillon-tutoriel` supprime intégralement le compte demo, sa base et le PDF publié ; une réinstallation repart proprement à zéro.
- **utils/peupler_demo.py** (script autonome, hors paquet) : alternative plus légère pour un jeu de données fictif (`communes_exemple`), sans tutoriel PDF :

```
bash python3 utils/peupler_demo.py --url https://<adresse-du-serveur> --admin-password '<mot de passe administrateur>'
```

Dans les deux cas, le mot de passe du compte demo est **fixe et documenté** (`demo`) — pratique pour une démonstration, mais à ne jamais laisser sur une VM exposée ou de production, contrairement au compte administrateur (mot de passe généré aléatoirement, §5).

Jeu de données massif (`sillon-demo-sirene`)

Sixième paquet `.deb` , optionnel, complémentaire de `sillon-tutoriel` (mêmes règles : jamais en production, dérogation assumée). Il télécharge et importe le fichier Sirene « StockEtablissement »

complet (INSEE, ~43,9 millions de lignes, Licence Ouverte 2.0), puis dépose un second jeu de scripts Python/R démontrant les possibilités de l'environnement d'exécution à cette échelle (cahier des charges §12.8).

C'est le seul paquet du projet qui a besoin d'un accès Internet réel sur la machine cible au moment de son installation — tous les autres composants sont vendorisés (§4). Deux façons de procéder :

- **Automatique** (cas courant, VM disposant d'une sortie Internet) :

```
bash sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb
```

Le `postinst` télécharge lui-même l'archive (environ 2,9 Go), l'extrait et la réduit aux colonnes utiles (environ 3,8 Go de CSV, quelques heures selon la machine — la sortie du `postinst` progresse par paliers de 5 millions de lignes), puis l'importe via l'API d'import normale.

- **Manuelle** (machine cible sans sortie Internet directe, ou pour ne pas dépendre de la durée du téléchargement pendant l'installation) : télécharger au préalable, depuis une machine qui a accès à Internet, le fichier « Sirene : Fichier StockEtablissement » (pas la variante *Historique*, pas le format *parquet*) depuis data.gouv.fr (jeu de données « Base Sirene des entreprises et de leurs établissements »), puis :

```
bash sudo mkdir -p /var/lib/sillon-demo-sirene sudo cp stock-stocketablissement-csv.zip /var/lib/sillon-demo-sirene/stock_etablissement.zip sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb
```

Le `postinst` détecte l'archive déjà présente et saute le téléchargement, pour passer directement à l'extraction puis à l'import. Le même mécanisme de reprise s'applique si une précédente tentative a déjà produit le fichier réduit (`sirene_reduit.csv` dans ce même répertoire) : `dpkg -i` peut être relancé sans redemander le fichier ni recommencer l'extraction en cas d'échec à une étape ultérieure (constaté en pratique lors des tests de ce paquet).

Prévoir au moins **12 Go d'espace disque libre** sur la partition qui héberge `/var/lib` le temps de l'installation (archive + fichier réduit + tampon d'import), même si l'espace définitivement occupé par la table importée est moindre.

`sudo apt-get purge sillon-demo-sirene` ne supprime que son répertoire de travail temporaire : la donnée elle-même (une table de plus dans la base du compte demo) disparaît avec `sillon-tutoriel`, quel que soit l'ordre de purge entre les deux paquets.

7.3 Quotas et paramètres applicatifs

Les quotas (taille maximale d'import CSV, durée maximale d'un job, CPU/RAM/PID par conteneur, quota disque par base, etc.) sont stockés dans la table `parametres` du catalogue applicatif, modifiable sans redémarrage de service ni reconstruction de paquet — soit depuis le panneau d'administration de l'application, soit directement en SQL :

```
UPDATE public.parametres SET valeur = '4096' WHERE cle = 'taille_max_csv_mo';
```

Valeurs par défaut à l'installation :

Paramètre	Valeur par défaut
<code>taille_max_csv_mo</code>	2048
<code>seuil_import_synchrone_mo</code>	10
<code>duree_max_job_minutes</code>	30
<code>cpu_max_conteneur_vcpu</code>	2
<code>ram_max_conteneur_mo</code>	4096
<code>jobs_simultanes_par_utilisateur</code>	1
<code>quota_disque_base_mo</code>	20480
<code>work_mem_defaut_mo</code>	64
<code>maintenance_work_mem_defaut_mo</code>	256
<code>connexions_max_par_utilisateur</code>	5

Import volumineux lent à indexer : chaque import CSV crée automatiquement un index par colonne (§7.4 du cahier des charges — GIN trigram pour le texte, B-tree pour les dates), coûteux en mémoire de travail pour un fichier de plusieurs millions de lignes. `maintenance_work_mem_defaut_mo` (appliqué par utilisateur, comme `work_mem_defaut_mo`) contrôle cette mémoire — la relever nettement (par exemple 1024 à 2048 selon la RAM disponible sur le serveur) accélère sensiblement la construction des index pour un import de cette taille :

```
UPDATE public.parametres SET valeur = '2048' WHERE cle =
'maintenance_work_mem_defaut_mo';
ALTER ROLE "<role_pg_de_l_utilisateur>" SET maintenance_work_mem = '2097152'; -- en
kilo-octets, 2048 * 1024
```

Cette seconde commande ne s'applique qu'aux comptes déjà créés (`role_pg` visible dans `public.utilisateurs`) : modifier le paramètre seul suffit pour tout compte créé après coup. `sillon-demo-sirene` applique déjà cette optimisation automatiquement à son installation pour le compte de démonstration (§7.2).

Identifier les requêtes les plus coûteuses : `sillon-server` active `pg_stat_statements` à l'installation (§7.1) — consultable directement en SQL, par exemple pour les dix requêtes cumulant le plus de temps d'exécution :

```
SELECT query, calls, round(total_exec_time) AS temps_total_ms, round(mean_exec_time)
AS temps_moyen_ms
FROM public.pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

8. Désinstallation

Deux niveaux, sémantique standard `dpkg` :

- **remove** : arrête les services et retire les fichiers du paquet, mais conserve `/etc/sillon/secrets.env`, `/etc/sillon-api.conf` et la base `sillon_catalog` — une réinstallation ultérieure reprend alors à l'identique (mêmes secrets, mêmes comptes), sans passer par la branche « premier déploiement ».

```
bash sudo apt-get remove sillon-image-execution sillon-worker sillon-orchestrateur  
sillon-server
```

- **purge** : en plus de ce qui précède, supprime réellement le catalogue applicatif (base `sillon_catalog`, rôles PostgreSQL personnels et de service qu'elle porte) ainsi que `/etc/sillon`, `/etc/sillon-api.conf`, `/etc/ssl/sillon` et `/var/lib/sillon` — une réinstallation ultérieure repart alors intégralement à zéro (nouveaux secrets, nouveau compte administrateur).

```
bash sudo apt-get purge sillon-image-execution sillon-worker sillon-orchestrateur  
sillon-server
```

Dans les deux cas, **les bases de travail créées par les agents** (§4.4 du cahier des charges — une base physique par agent, distincte du catalogue `sillon_catalog`) **ne sont jamais supprimées automatiquement**, ni par `remove` ni par `purge` : ce sont des données applicatives, pas des artefacts d'installation. Pour les retirer explicitement si c'est réellement souhaité :

```
sudo -u postgres psql -c "\l" | grep sillon_  
sudo -u postgres dropdb <nom_de_la_base>
```

Cas particulier de `sillon-tutoriel` : contrairement aux quatre autres paquets, son `purge` supprime intégralement le compte `demo@sillon.local`, sa base et le PDF publié — c'est un compte de démonstration jetable, pas une donnée d'agent réel, la même prudence ne s'applique pas.

```
sudo apt-get purge sillon-tutoriel
```

Rappel : aucun mécanisme de sauvegarde n'est porté par l'application — la protection contre la perte de données relève de l'infrastructure d'hébergement de la direction.

9. Dépannage courant

Symptôme	Cause probable	Vérification
Import CSV volumineux rejeté (413)	Anciennement un défaut Nginx (<code>client_max_body_size</code>) — corrigé, la limite Nginx est illimitée et seule la limite applicative (<code>taille_max_csv_mo</code>) s'applique	<code>grep</code> <code>client_max_body_size</code> <code>/etc/nginx/sites-available/sillon</code>
Script utilisateur ne démarre jamais	Session D-Bus de <code>www-data</code> non encore prête après un redémarrage du serveur	<code>systemctl status</code> <code>user@33.service</code> , puis <code>systemctl restart</code> <code>sillon-worker</code>
<code>sillon-orchestrateur</code> refuse de s'installer	<code>sillon-server</code> non installé/configuré au préalable (<code>/etc/sillon/secrets.env</code> absent)	Installer/vérifier <code>sillon-server</code> d'abord
Requêtes SQL normales, mais tous les scripts échouent (statut « erreur ») après un changement d'adresse IP ou un redémarrage du serveur	Adresse hôte détectée par le <code>postinst</code> de <code>sillon-worker</code> obsolète (§9.1)	<code>journalctl -u</code> <code>sillon-worker -n 50</code> , rechercher une erreur de connexion PostgreSQL
Bouton « Journal » (onglet Suivi) affiche systématiquement « journal vide pour l'instant », même pour un job en cours	Corrigé (<code>python3 -u / stdbuf</code> , <code>worker.py</code>) : la sortie standard d'un interprète non attaché à un terminal était mise en tampon par blocs, jamais vidée avant la fin d'un script court — le journal réellement produit n'apparaissait qu'une fois le job déjà terminé (et donc déjà uniquement dans l'archive téléchargeable, plus dans la fenêtre « en cours »)	<code>sudo dpkg -i sillon-worker_0.1.0_all.deb</code> puis <code>systemctl restart</code> <code>sillon-worker</code> si une version antérieure au correctif est installée

9.1 Changement d'adresse IP du serveur (ou redémarrage avec IP dynamique)

Symptôme observé en pratique : après un changement d'adresse IP (DHCP, réaffectation de VM, changement d'interface réseau) suivi ou non d'un redémarrage, les requêtes SQL de l'onglet Travaux continuent de fonctionner normalement, mais **tous** les scripts Python/R déposés échouent immédiatement (statut « erreur », sans même produire de journal d'exécution).

Cause : les scripts s'exécutent dans un conteneur Podman rootless, qui n'a pas d'accès direct à `localhost` de l'hôte. Le réseau rootless par défaut (`pasta`) route l'alias `host.containers.internal` vers l'hôte réel, mais en réémettant la connexion avec l'adresse IP LAN de l'hôte comme adresse source (jamais `127.0.0.1`, constaté en pratique dans les journaux PostgreSQL) — c'est pourquoi le `postinst` de `sillon-worker` autorise explicitement **cette seule adresse** dans `pg_hba.conf`, détectée une fois à l'installation :

```
host    all                    all                    <adresse détectée à l'installation>/32
scram-sha-256
```

Cette détection n'est **jamais refaite automatiquement** par la suite. Si l'adresse IP du serveur change après coup, cette règle devient obsolète : PostgreSQL rejette toute connexion venant de la nouvelle adresse, donc tout script (qui doit s'y connecter via `SILLON_DSN`) échoue dès sa tentative de connexion — alors que les requêtes SQL de l'onglet Travaux, elles, transitent par l'orchestrateur en `host=localhost` et ne sont jamais concernées.

Remédiation — deux options équivalentes :

- **Réinstaller `sillon-worker`** (le plus simple, redétecte l'adresse courante et ajoute la règle manquante sans toucher au reste de la configuration) :

```
bash sudo dpkg -i sillon-worker_0.1.0_all.deb
```

L'ancienne règle (adresse obsolète) n'est pas supprimée — elle ne gêne pas, `pg_hba.conf` accepte plusieurs règles `host` sans conflit — mais peut être retirée manuellement par propreté si souhaité.

- **Corriger à la main**, sans reconstruire/redéployer de paquet :

```
bash ADRESSE_HOTE=$(ip -4 route get 1.1.1.1 | grep -oP 'src \K\S+' | head -1) echo "host
all all ${ADRESSE_HOTE}/32 scram-sha-256" \ | sudo tee -a
/etc/postgresql/17/main/pg_hba.conf sudo systemctl reload postgresql
```

Point de vigilance pour un serveur à adresse IP dynamique (DHCP sans bail réservé) : ce mécanisme n'est pas auto-réparant — chaque changement d'adresse casse à nouveau les scripts jusqu'à la prochaine réinstallation ou correction manuelle. Pour un déploiement durable, préférer une **adresse IP statique ou un bail DHCP réservé** pour le serveur SILLON, ce qui élimine le problème à la source plutôt que de le corriger à chaque occurrence.

Autres points de friction possibles liés à un changement d'adresse IP (moins critiques, sans impact constaté à ce jour) :

- Le certificat TLS auto-signé est généré avec `CN=localhost` (aucune adresse IP ni nom d'hôte spécifique) : un changement d'IP ne l'invalide pas, mais l'avertissement de certificat non reconnu par le navigateur reste présent quelle que soit l'adresse utilisée pour se connecter — sans lien avec ce changement.
- Si `SILLON_URL` a été configuré manuellement (`systemctl edit sillon-orchestrateur` , §7.1) avec une adresse IP littérale plutôt qu'un nom de domaine, les liens inclus dans les notifications par mail pointeront vers l'ancienne adresse après un changement — à corriger manuellement dans ce cas précis.